



Programação Funcional: do Cálculo Lambda à Implementação de Métodos Numéricos

Enzo Rocha Leite Diniz Ribas Luis A. D'Afonseca Lucas Martins Rocha

Centro Federal de Educação Tecnológica de Minas Gerais
Belo Horizonte – MG

22ª Semana de Ciência & Tecnologia do CEFET-MG



Objetivo da apresentação

Pergunta central

Como ideias de programação funcional ajudam a escrever, testar e raciocinar sobre métodos numéricos?

- 1 Funções matemáticas e funções em programas.
- 2 Cálculo lambda como modelo formal de aplicação de funções.
- 3 Composição, erros explícitos e iteração numérica.
- 4 Newton–Raphson como exemplo principal.



Da função matemática ao programa

Na matemática, uma função associa entradas válidas a saídas:

$$f : \mathbb{R} \longrightarrow \mathbb{R}, \quad f(x) = \sqrt{x}. \quad (1)$$

Entrada	Resultado em $\mathbb{R}$
4	2
0	0
-1	não definido em $\mathbb{R}$

Problema computacional

Como o programa deve representar uma entrada fora do domínio?



Imperativo: erro fora da assinatura

```
double sqrt(double x) {  
    if (x < 0)  
        throw new IllegalArgumentException();  
    return Math.sqrt(x);  
}
```

- A assinatura sugere apenas: `double -> double`.
- A exceção existe, mas aparece fora do fluxo normal do valor.
- O chamador precisa conhecer esse comportamento por documentação, teste ou leitura do código.

Ideia funcional

Tornar o caso inválido parte explícita do resultado.



Funcional: erro como valor

Em vez de esconder a falha em uma exceção, podemos modelar:

$\text{safeSqrt} : \mathbb{R} \longrightarrow \text{Maybe } \mathbb{R}.$

$$\text{safeSqrt}(x) = \begin{cases} \text{Just}(\sqrt{x}), & x \geq 0, \\ \text{Nothing}, & x < 0. \end{cases}$$

Entrada	Resultado
4	Just 2
0	Just 0
-1	Nothing



Cálculo lambda: três ideias

O cálculo lambda é um modelo formal para representar funções e computações.

Ideia	Forma	Leitura
Abstração	$\lambda x. e$	função que recebe x e calcula e
Aplicação	$e_1 e_2$	aplica e_1 ao argumento e_2
Redução	$(\lambda x. e) v \rightarrow e[x := v]$	substitui x por v em e

Exemplo:

$$\lambda x. x^2$$

representa a função que recebe x e devolve x^2 .



Beta Reduction

Considere a função que soma 1 ao argumento:

$$(\lambda x. x + 1) 4.$$

A aplicação é avaliada por substituição:

$$(\lambda x. x + 1) 4 \longrightarrow 4 + 1 \longrightarrow 5.$$

Outro exemplo, com função quadrática:

$$(\lambda x. x \cdot x + 1) 3 \longrightarrow 3 \cdot 3 + 1 \longrightarrow 10.$$

No contexto de métodos numéricos

Uma iteração numérica também aplica uma transformação a uma aproximação atual para produzir a próxima.



Funções de Alta Ordem (Funções que recebem e retornam funções)

Em cálculo lambda, funções podem ser argumentos de outras funções.

$$\text{aplicarDuasVezes} = \lambda g. \lambda x. g(g(x)).$$

Se $g(x) = x + 1$:

$$(\lambda g. \lambda x. g(g(x))) (\lambda y. y + 1) 3 \rightarrow 5.$$

Ideia importante

Métodos iterativos seguem exatamente essa estrutura: aplicar repetidamente uma função de atualização.



Currying e especialização

Uma função de dois argumentos pode ser vista como uma função que retorna outra função:

$$f(x, y) = x + y \quad \Longleftrightarrow \quad f = \lambda x. \lambda y. x + y.$$

Fixando $x = 10$:

$$\text{somar10} = f(10) = \lambda y. 10 + y.$$

Logo:

$$\text{somar10}(5) = 15.$$

Uso prático

Currying permite criar funções especializadas a partir de funções gerais.



Composição com contexto

Algumas operações possuem restrições de domínio:

$$x \xrightarrow{\sqrt{}} \sqrt{x} \xrightarrow{\log} \log(\sqrt{x}).$$

Usando Maybe, cada etapa informa se produziu valor ou falhou:

$$\text{Maybe } A \longrightarrow (A \longrightarrow \text{Maybe } B) \longrightarrow \text{Maybe } B.$$

Entrada	Resultado de $\log(\sqrt{x})$
4	<code>Just(log 2)</code>
0	<code>Nothing</code> , pois $\log(0)$ é inválido
-1	<code>Nothing</code> , pois $\sqrt{-1}$ é inválido



O que é programação funcional?

A programação funcional trata **funções como abstrações centrais da computação**.

Em vez de organizar o programa principalmente por comandos e mutações de estado, ela organiza a computação por **transformações de dados**.

- funções de primeira classe;
- funções puras;
- imutabilidade.
- composição;
- recursão;
- efeitos controlados.



Funções de primeira classe

Uma função é de primeira classe quando pode ser tratada como um valor.

Isso significa que ela pode ser:

- armazenada em uma variável;
- passada como argumento;
- retornada por outra função.

Ligação com métodos numéricos

Um método de Newton genérico recebe f e f' como argumentos.

$$\text{newtonStep}(f, f', x) = x - \frac{f(x)}{f'(x)}.$$



Funções puras

Uma função pura depende apenas dos argumentos de entrada.

Definição prática

Para a mesma entrada, produz sempre a mesma saída e não altera o estado observável do programa.

$$f : \mathbb{R} \longrightarrow \mathbb{R}, \quad f(x) = x^2 + 1. \quad (2)$$

A chamada da função pode ser entendida como uma transformação matemática:

$$x \xrightarrow{f} f(x).$$



Efeito colateral

Com efeito colateral:

```
int counter = 0;

int f(int x) {
    counter++;
    return x * x;
}
```

O que acontece?

Além de retornar x^2 , a função altera counter. Portanto, seu comportamento não é descrito apenas por $x \mapsto x^2$.



Função pura

Sem efeito colateral:

```
int f(int x) {  
    return x * x;  
}
```

$$f : x \mapsto x^2.$$

Consequência

A função fica mais fácil de testar, compor e reproduzir.



Imutabilidade

Imutabilidade significa evitar sobrescrever valores já criados.

Em um método iterativo, isso combina bem com a própria notação matemática:

$$x_0 \longrightarrow x_1 \longrightarrow x_2 \longrightarrow \dots$$

Cada aproximação é um novo valor, não a “mesma variável” modificada.

Leitura numérica

Em vez de pensar apenas em $x = \dots$, pensamos em uma sequência $x_0, x_1, x_2, \dots$



Composição

Composição é construir uma função a partir de outras funções.

$$f : A \longrightarrow B, \quad g : B \longrightarrow C \quad \Rightarrow \quad g \circ f : A \longrightarrow C. \quad (3)$$

A aplicação composta é:

$$(g \circ f)(x) = g(f(x)).$$

Em métodos numéricos

Podemos separar cálculo do passo, validação, critério de parada e atualização.



Recursão

Recursão descreve uma computação em termos dela mesma.

Um exemplo clássico é o fatorial:

$$n! = \begin{cases} 1, & n = 0, \\ n \cdot (n - 1)!, & n > 0. \end{cases}$$

Ligação com iteração

Métodos numéricos também podem ser vistos recursivamente:

$$x_{n+1} = N(x_n).$$



Efeitos controlados

Nem toda computação é uma função matemática pura.

Programas reais lidam com:

- erros;
- entrada e saída;
- estado;
- valores ausentes;
- falhas de convergência.

Ideia funcional

O efeito não precisa ficar escondido. Ele pode aparecer no tipo ou no resultado.



Efeitos controlados em métodos numéricos

Em Newton–Raphson, nem toda execução converge.

Em vez de retornar apenas um número, o algoritmo pode retornar um resultado estruturado:

```
NewtonResult = Converged | MaxIterations | InvalidDerivative | ...
```

Ponto central

A falha deixa de ser um comportamento escondido e passa a fazer parte da interface do método.



Ponte com métodos numéricos

Considere a função de atualização de Newton:

$$N_{f,f'}(x) = x - \frac{f(x)}{f'(x)}. \quad (4)$$

A iteração é a aplicação repetida dessa função:

$$x_{n+1} = N_{f,f'}(x_n). \quad (5)$$

funções de primeira classe $\rightarrow$ função pura $\rightarrow$ composição $\rightarrow$ iteração



Métodos numéricos como transformações

Métodos numéricos são algoritmos construídos a partir de transformações matemáticas.
Para Newton–Raphson:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (6)$$

Definindo a função de atualização:

$$N_f(x) = x - \frac{f(x)}{f'(x)}, \quad (7)$$

a sequência de aproximações é:

$$x_0, \quad N_f(x_0), \quad N_f(N_f(x_0)), \quad N_f^3(x_0), \dots$$

Perspectiva funcional

A iteração é aplicação repetida de uma função.



Exemplo numérico: raiz de $x^2 - 2$

Queremos aproximar $\sqrt{2}$ resolvendo:

$$f(x) = x^2 - 2, \quad f'(x) = 2x.$$

A função de Newton é:

$$N_f(x) = x - \frac{x^2 - 2}{2x} = \frac{x}{2} + \frac{1}{x}.$$

Começando com $x_0 = 1$:

Iteração	Aproximação
x_0	1.0000
$x_1 = N_f(x_0)$	1.5000
$x_2 = N_f(x_1)$	1.4167
$x_3 = N_f(x_2)$	1.4142



Newton: implementação funcional

Um passo de Newton pode ser escrito como uma função pura:

```
newtonStep f df x = x - f x / df x
```

Para $f(x) = x^2 - 2$:

```
f x = x*x - 2
```

```
df x = 2*x
```

```
x1 = newtonStep f df x0
```

Leitura matemática

`newtonStep` recebe f , f' e x_n , devolvendo x_{n+1} .



Falhas também fazem parte do método

Newton–Raphson pode falhar por razões matemáticas ou computacionais:

- derivada nula;
- domínio inválido;
- divergência;
- máximo de iterações;
- tolerância não atingida;
- perda de precisão.

Uma saída mais informativa é:

`NewtonResult = Converged | MaxIterations | InvalidDerivative | ...`

Ponto central

O algoritmo não precisa fingir que todo resultado é apenas um número.



Síntese

- 1 Funções puras aproximam o programa da notação matemática.
- 2 Cálculo lambda formaliza abstração, aplicação e substituição.
- 3 Currying e funções de alta ordem permitem especialização.
- 4 Contextos como Maybe tornam falhas explícitas.
- 5 Métodos numéricos podem ser mais naturalmente modelados como composição e aplicação repetida de funções.

Mensagem final

A programação funcional não substitui a imperativa, mas oferece ferramentas para estruturar problemas matemáticos de forma mais natural e segura.



Contato

Enzo Rocha Leite Diniz Ribas

- GitHub: @oenzoribas
- LinkedIn: Enzo Ribas
- E-mail:
enzorochaleitedinizribas@gmail.com
- Instagram: ribas_enzo



Agradecimentos

Do Cálculo Lambda à Programação Funcional.

Thanks for your attention.



Referências



PIERCE, B. C.
Types and Programming Languages.
MIT Press, 2002.



HUTTON, G.
Programming in Haskell.
Cambridge University Press.



CHAPRA, S. C.; CANALE, R. P.
Numerical Methods for Engineers.
McGraw-Hill Education.



CHURCH, A.
An Unsolvable Problem of Elementary Number Theory.
American Journal of Mathematics, 1936.



BURDEN, R. L.; FAIRES, J. D.
Numerical Analysis.
Cengage Learning.